ShortP2P Protocol

Langue: French / Français

Author: Igor Evdokimov

Repository: https://github.com/IskraMessenger/ShortP2P

Created: 2026-11-11

Modified: 2026-11-11

Related: docs/encryption, docs/messages, protocol.md

Sommaire

Page de titre

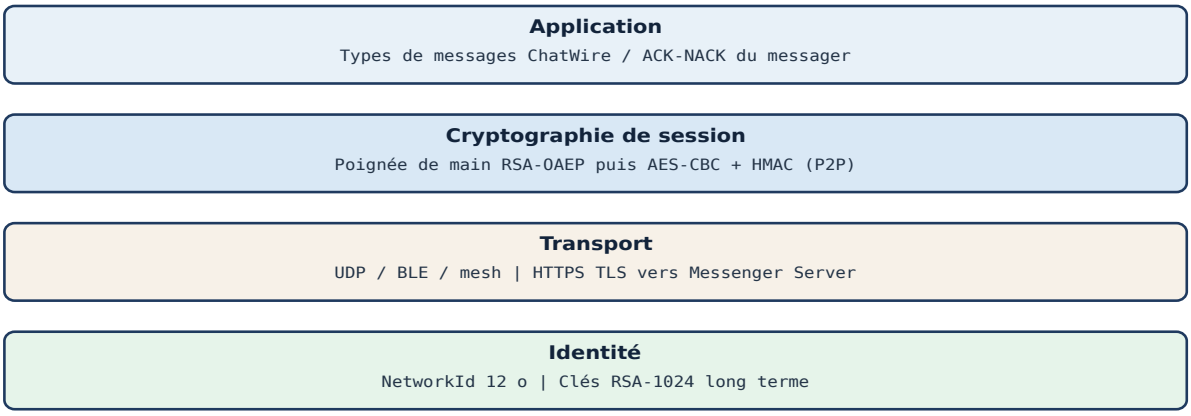
Sommaire

Schémas communs

Description du protocole

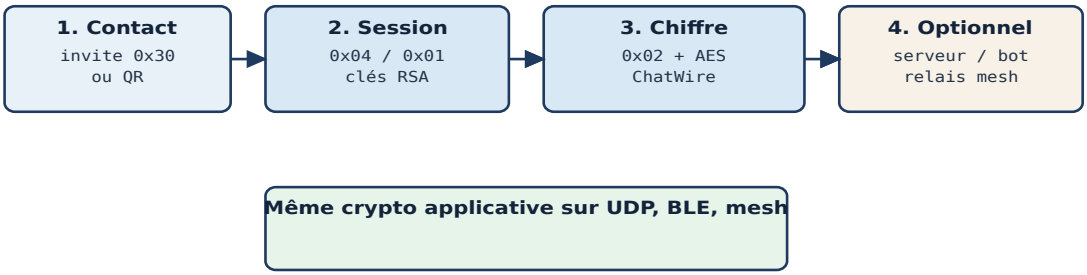
Schémas communs

Couches du protocole



Pile logique (pas des couches OS). Détails : docs/encryption, docs/messages.

Flux type d'un chat direct



Ports UDP par défaut



Bluetooth et mesh réutilisent les mêmes trames ; HTTPS pour Messenger Server.

Description du protocole

ShortP2P est une pile de messagerie entre pairs. En mots simples : deux appareils (pairs) se trouvent, se mettent d'accord sur des clés secrètes, puis échangent des messages de chat que personne au milieu ne peut lire. Le transport peut être direct (UDP, Bluetooth, radio mesh/LoRa) ou passer par un Messenger Server facultatif qui ne relaie que des données déjà chiffrées.

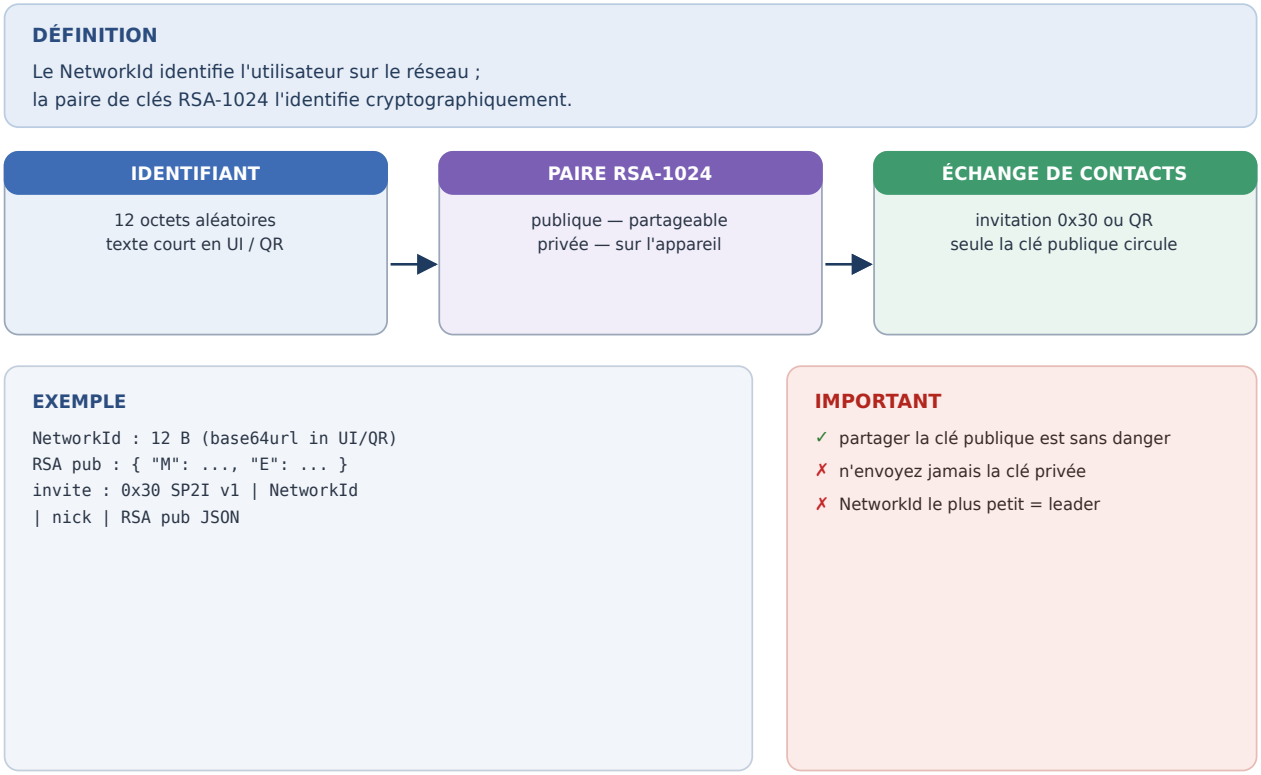
NOTIONS DE BASE (aucune connaissance en cryptographie requise)

- Pair (peer) : l'autre appareil / la personne avec qui vous discutez.
- NetworkId : 12 octets aléatoires identifiant un utilisateur, comme une adresse réseau. Dans l'interface et les QR codes, il apparaît en texte base64url court (~16 caractères).
- RSA-1024 : un algorithme asymétrique — une paire de clés. La clé publique se partage ouvertement (invitations, QR codes) ; la clé privée ne quitte jamais l'appareil. Ce qui est chiffré avec la clé publique ne peut être déchiffré qu'avec la clé privée correspondante. Dans ShortP2P, on l'utilise une fois par session pour envoyer les secrets de session au pair.
- Clés de session : deux chaînes d'octets aléatoires créées pour une session de chat : une clé AES (16 octets) et une clé MAC (32 octets). Après la poignée de main, les deux côtés possèdent les mêmes deux chaînes.
- AES-128-CBC : un chiffrement symétrique — la même clé chiffre et déchiffre. Le mode CBC enchaîne les blocs pour que deux fragments identiques paraissent différents ; le remplissage PKCS7 aligne la longueur sur un multiple de 16 octets.
- IV (vecteur d'initialisation) : 16 octets aléatoires neufs pour chaque paquet, afin que le même texte produise un texte chiffré différent à chaque envoi.
- Étiquette HMAC-SHA256 : une somme de contrôle avec clé (16 octets utilisés) calculée sur IV + texte chiffré. Le destinataire recalcule et compare en temps constant ; une différence signifie un paquet altéré ou corrompu, il est rejeté.
- ChatWire : le format binaire propre de ShortP2P pour le contenu du chat. C'est lui qui est chiffré et envoyé ; le destinataire déchiffre d'abord, puis l'analyse.
- S2P1 : une « signature magique » de 4 octets (ASCII "S2P1") au début de chaque trame ChatWire. Elle sert d'en-tête : la première chose que l'analyseur vérifie pour reconnaître le format.
- Octet de type (kind) : l'octet juste après la magie. Sa valeur indique quel type de message suit.
- ACK / NACK : rapports internes de livraison. ACK = « message entièrement reçu » ; NACK = « renvoyez les morceaux portant ces numéros ».
- TLS/HTTPS : le chiffrement de transport standard entre le client et le Messenger Server (le cadenas des navigateurs).
- JWT : un jeton signé remis par le serveur à la connexion. Le client le joint à chaque requête ; le serveur vérifie la signature au lieu de redemander le mot de passe.
- Chiffrement de bout en bout (E2E) : seuls les deux appareils qui discutent peuvent lire le contenu. Le Messenger Server le stocke et le relaie comme un bloc chiffré

opaque (texte Base64) et ne peut pas le lire.

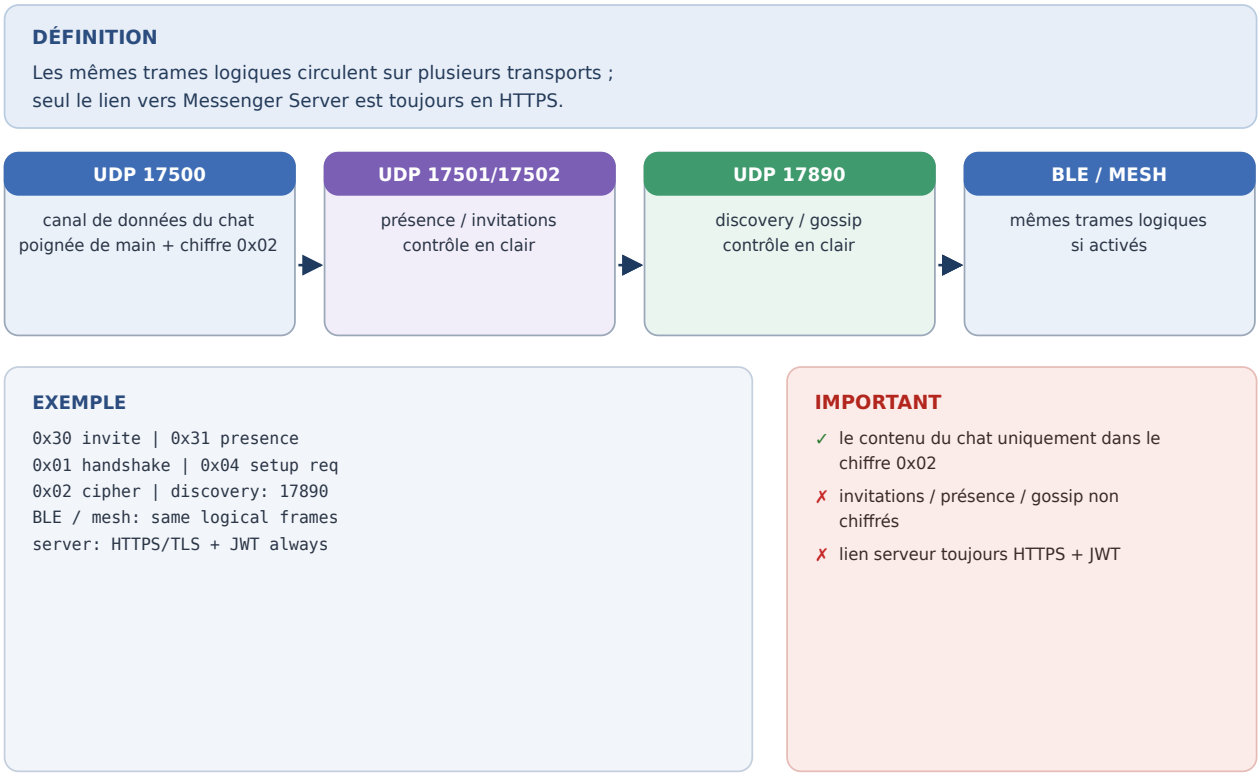
- Bot : un compte programme sur le serveur avec lequel les clients peuvent parler ; lui aussi ne reçoit que du contenu chiffré.
- Gossip / discovery : le trafic annexe qui répond à « qui est là et comment l'atteindre » (UDP 17890).
- RELAY : une trame de routage LAN capable de transporter une charge déjà chiffrée via des nœuds intermédiaires.

1) IDENTITÉ



Chaque utilisateur possède un NetworkId (12 octets aléatoires) et une paire RSA-1024 créée à l'inscription. La clé publique circule en clair dans les invitations (0x30) ou les QR codes — c'est sans danger, une clé publique est faite pour être partagée. Règle : le pair au NetworkId le plus petit devient le « leader » de la poignée de main ; l'autre attend.

2) TRANSPORTS

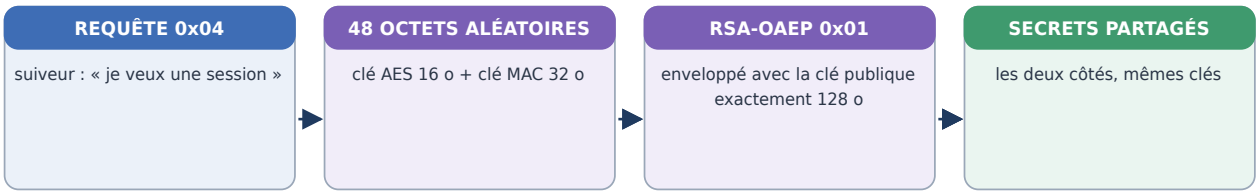


Ports UDP par défaut : 17500 — canal de données du chat (poignée de main et paquets chiffrés), 17501 — annonces de présence, 17502 — invitations, 17890 — discovery/gossip. Bluetooth et mesh/LoRa transportent les mêmes trames logiques quand ils sont actifs. La communication avec le Messenger Server passe toujours par HTTPS/TLS avec authentification JWT.

3) SESSION (P2P direct), étape par étape

DÉFINITION

Le leader envoie deux secrets aléatoires enveloppés dans la clé publique du pair ; ensuite les deux côtés chiffrent avec la même clé AES.



EXEMPLE

```
0x04 || NetworkId(12)
0x01 || RSA-OAEP-SHA1(128) // keys 48 B
0x02 || IV(16) || AES-CBC || tag(16)
packet <= 1024 B | plain <= 991 B
```

IMPORTANT

- ✓ vérifier l'étiquette HMAC avant de déchiffrer
- ✗ le leader ignore un 0x01 étranger
- ✗ IV aléatoire neuf pour chaque paquet

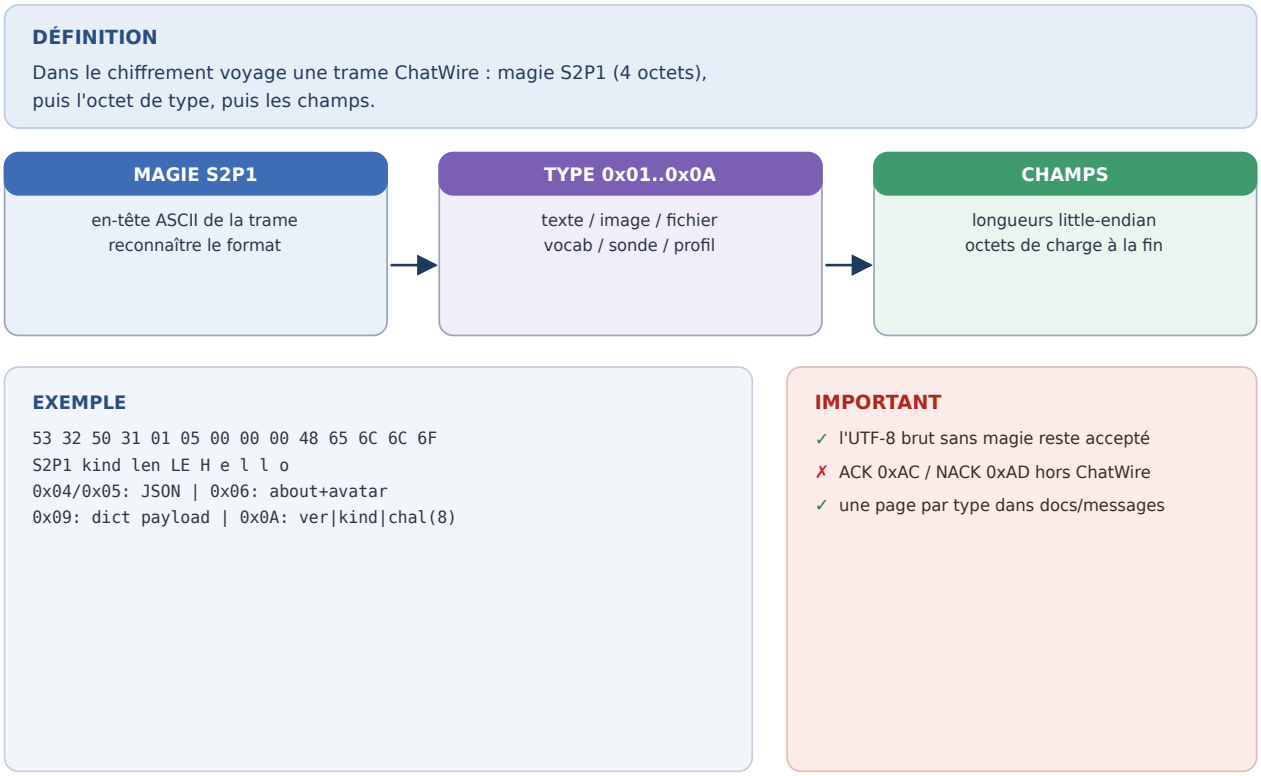
a. Le suiveur peut envoyer la trame 0x04 avec son NetworkId : « je veux une session ».

b. Le leader génère deux secrets aléatoires (clé AES 16 o + clé MAC 32 o) et chiffre ce paquet de 48 octets avec la clé publique RSA du pair via RSA-OAEP(SHA-1). Le résultat fait exactement 128 octets et part en trame 0x01.

c. Le suiveur déchiffre avec sa clé privée RSA ; les deux côtés partagent alors les mêmes secrets.

d. Chaque trame de données suivante est 0x02 + paquet, où paquet = IV(16) + texte chiffré AES-CBC + étiquette HMAC(16), au plus 1024 octets. Le destinataire vérifie d'abord l'étiquette, puis déchiffre.

4) MESSAGES APPLICATIFS (dans le chiffrement)



Le texte clair passé au chiffrement est généralement une trame ChatWire : magie "S2P1", puis l'octet de type, puis les champs (longueurs en little-endian sauf mention contraire).

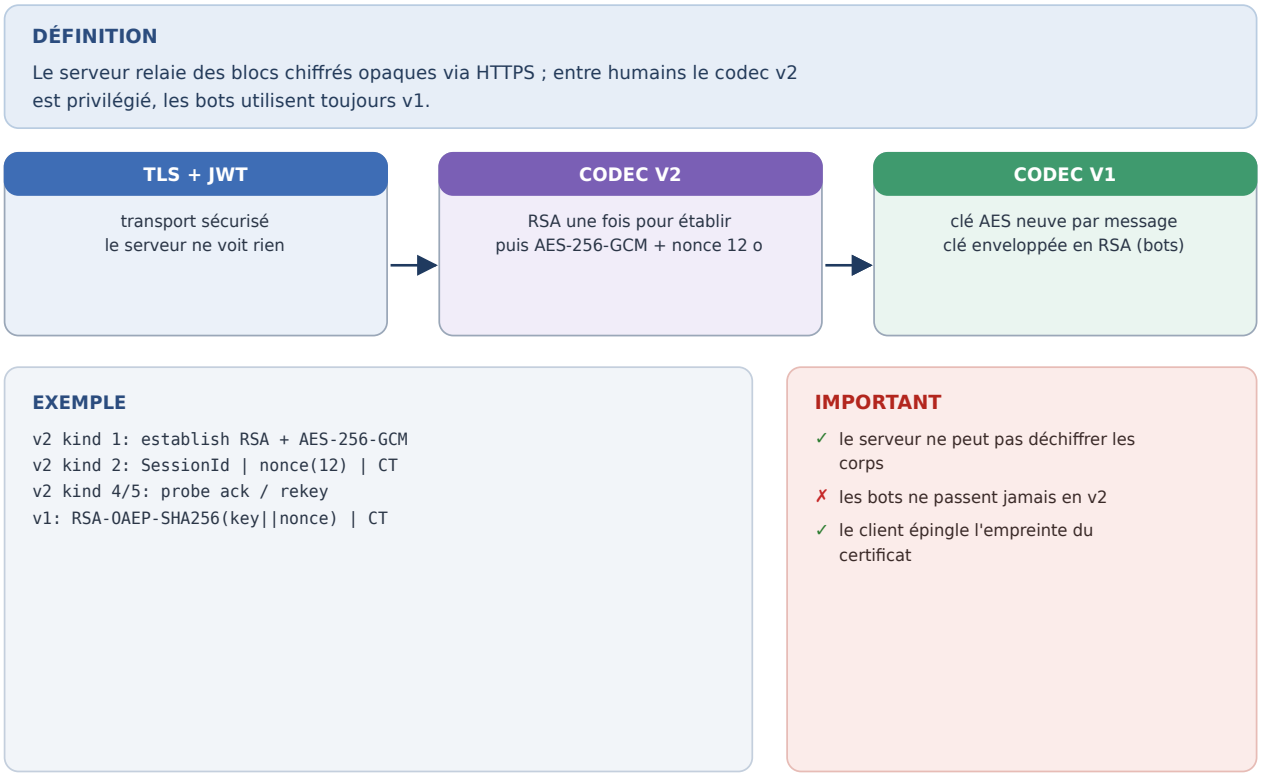
Valeurs de l'octet de type, en détail :
- 0x01 Text — un simple message texte (UTF-8), ce qu'une personne tape.
- 0x02 Image — une image dans le chat : type MIME (ex. "image/png") et octets de l'image.
- 0x03 File — un petit fichier envoyé dans le chat : nom, type MIME, octets.
- 0x04 Transfer offer — un JSON décrivant un gros fichier proposé au téléchargement hors du chat (hôte/port, jeton à usage unique, taille, expiration).
- 0x05 Transfer control — la réponse JSON à une offre : accepter, refuser, ou erreur.
- 0x06 User info — le profil de l'expéditeur (texte « à propos » et avatar), non affiché comme bulle de chat.
- 0x07 Vocabulary offer — la liste des dictionnaires de compression que l'expéditeur gère.
- 0x08 Vocabulary accept — le choix des dictionnaires pour ce chat (ou "UNCOMPRESSED").
- 0x09 Compressed text — un texte encodé avec le dictionnaire choisi pour économiser de la bande passante (utile sur radio lente).
- 0x0A Session probe — un petit « ping » dans le canal chiffré pour confirmer que les deux bouts déchiffrent correctement.

Hors ChatWire, mais eux aussi chiffrés : 0xAC ACK (message entièrement reçu, 17 octets)

et 0xAD NACK (renvoyer les numéros de morceaux listés). Les très vieux clients envoient de l'UTF-8 brut sans la magie S2P1 ; le destinataire le traite quand même comme du texte.

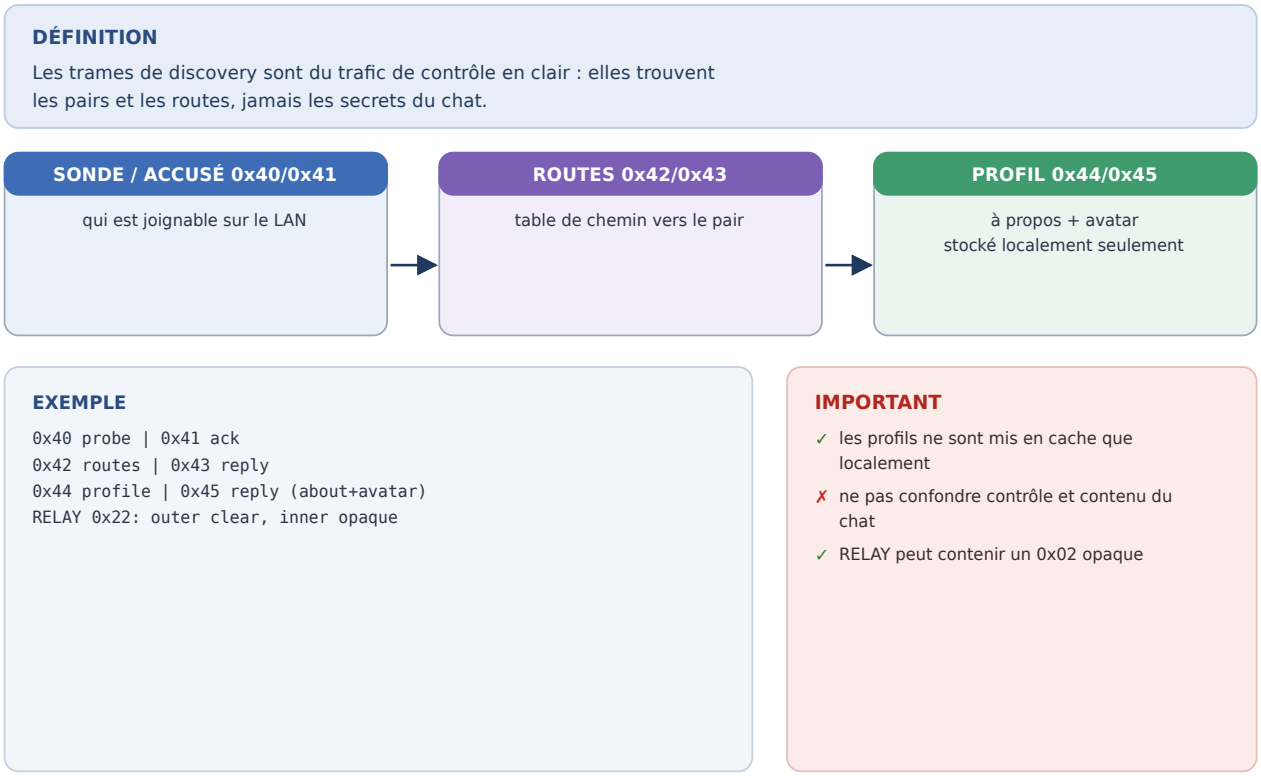
Une page par type avec structures d'octets et exemples non chiffrés : docs/messages. Les pipelines chiffrés en schémas : docs/encryption.

5) MESSENGER SERVER ET BOTS



Quand les pairs ne peuvent pas se joindre directement, ils passent par le Messenger Server en HTTPS/TLS+JWT. Le corps du chat est chiffré de bout en bout : le serveur ne stocke qu'un bloc Base64 opaque.
- Codec v2 (préféré entre humains) : RSA sert une fois pour établir la session ; ensuite chaque message est en AES-256-GCM (un chiffrement authentifié : chiffrement et contrôle d'intégrité en un seul algorithme) avec un nonce aléatoire de 12 octets par message.
- Codec v1 (pour les bots et en secours) : chaque message reçoit une clé AES-256-GCM et un nonce neufs, et la clé+nonce sont enveloppés avec la clé publique RSA du destinataire (RSA-OAEP-SHA256). Les bots utilisent toujours v1.
Dans les deux cas, le serveur ne peut pas déchiffrer le contenu.

6) DISCOVERY



Les trames 0x40..0x45 sur UDP 17890 (sondes/accusés, tables de routes, profils) sont surtout du trafic de contrôle en clair : elles servent à trouver les pairs et ne doivent pas être confondues avec le contenu secret du chat. Les trames LAN FIND/FOUND localisent l'adresse d'un pair ; une trame RELAY peut transporter une charge 0x02 déjà chiffrée via des nœuds intermédiaires.

POUR ALLER PLUS LOIN

- docs/encryption — les pipelines chiffrés en diagrammes vectoriels (établissement de session, formats de paquets, chemins serveur).
- docs/messages — une page par type de message avec exemples d'octets non chiffrés.
- protocol.md — référence détaillée des trames (en russe).